

SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering

例子 2：浏览器里找信息（网页操作）

人类 UI

看到网页 → 滚动 → 找按钮 → 点开 → 复制一段文字

ACI（两种常见形式）

形式 A：文本化网页（推荐）

- 观察：网页被转换成“可读文本 + 元素结构（标题/链接/表单）”
- 动作：`click(link_id)`、`type(input_id, "xxx")`、`scroll()`
优点：不怕网页布局变化，不会“点歪”。

形式 B：截图 + 坐标点击（更像真的点屏幕）

- 观察：截图
- 动作：点击某个坐标 `(x,y)`
缺点：网页改一点点布局，坐标就废了；不稳定。

所以 ACI 的核心价值之一：把“视觉 UI”转换成“结构化可操作对象”。

好一个将视觉UI转换成这个结构化可操作的对象

跟你具身智能方向的对应关系（你会秒懂）

具身智能里：

- 机器人不能直接“想象手怎么动”，需要一个 **action space（动作空间）**：比如关节角度、末端位姿、抓取开合
- 还需要 **observation（观测）**：相机、力传感器、状态

软件世界里：

- ACI 就是智能体的 **动作空间（run/edit/click）**
- 终端输出/文件内容/网页结构就是它的 **观测**

哲学设计思想（顺便提一下）：

把一个强大的“脑子”（LLM）放进一个“被约束的身体”（ACI/动作空间）里，通过明确的感知-行动闭环，让系统更可靠、更可控、更能落地。

如果你愿意，下一步我可以把 SWE-agent 里的 ACI 具体长什么样（它到底给了哪些 actions/observations）用一张“输入输出流程图”给你画出来。

📄 📌 🔍 ↺ ...

好一个软件工程智能体，规范化操作就是这个ACI的功能

ACI就是提供了一个标准化操作我们电脑的规范

你关心的点	ACI (Agent-Computer Interface) 智能体-计算机接口	MCP (Model Context Protocol) 模型上下文协议
它解决什么问题	智能体如何稳定地“操作电脑/项目”（打开文件、编辑、跑命令、点网页）	智能体如何标准化地“接入外部能力”（数据库、检索、浏览器抓取、内部系统）
更像什么	给智能体的“鼠标+键盘+终端+文件系统 API”	给工具的“USB/Type-C 接口标准”
核心形式	<code>observe -> action -> observe</code> 的闭环（动作空间 + 观测空间）	“工具清单 + 参数 schema + 调用协议 + 返回结果”
典型输出	<code>run_command("pytest")</code> 、 <code>open_file()</code> 、 <code>edit_file(diff)</code>	<code>call_tool("youtube.search", {...})</code> 、 <code>call_tool("db.query", {...})</code>
典型场景	SWE-agent 修 bug、跑测试、改代码、提交 git	让模型调用别人写好的工具：抓网页、查知识库、拉数据

用一个具体例子把“配合关系”讲透（SWE-agent / Claude Code 都类似）

假设任务：“修复项目里某个测试失败，并查一下最新依赖版本”

ACI 在干什么（操作电脑本体）

- `run_command("pytest -q")` 看到失败日志
- `open_file("xxx.py")` 定位代码
- `edit_file(diff)` 改代码
- 再 `run_command("pytest -q")` 验证通过

这些都是“在本机项目里动手”的能力——典型 ACI。

MCP 在干什么（接外部工具）

当它需要“外部信息/外部系统”时：

- 调用一个 MCP 工具：`pip_check_latest(package)` 或 `web_search(query)`
- 或者调用你说的：`youtube抓取` / `frgm` 这种第三方 MCP server 提供的能力

这些是“通过协议去调用外部工具”——典型 MCP。

所以：ACI 像手脚；MCP 像插口/总线，帮你把各种外接器官插上去。



MCP是用来调用这个接口提供的能力，这个能力一般都是，比如说脚本实现

你之前问的那个点：抓 YouTube 数据还需要 MCP 吗？

- 如果你的智能体只是用浏览器像人一样点开 YouTube、复制粘贴：这是 **ACI**（但不稳定，容易被页面变化/登录/反爬搞死）。
- 如果你希望**稳定、结构化、可复用地抓取**（比如直接拿到视频列表/字幕/元数据）：用 **MCP** 接一个专门的 **YouTube 工具** 通常更靠谱。

现实里最常见的组合：**ACI 做“工程内操作”，MCP 做“外部数据/能力”**。

有意思，接一个专门用来调用youtube脚本是吗

通过mcp接上这个接口进行调用

Guardrails（护栏/安全边界）= 防止智能体乱来、把系统搞坏的一套约束机制。

你可以把它理解成：

给 agent 的“安全驾驶规则 + 自动刹车系统”。

因为 agent 在做 SWE 任务时，会：

- 改代码（可能把项目改崩）
- 跑命令（可能误删文件、泄露密钥）
- 操作 git（可能提交一堆垃圾改动）

所以需要 Guardrails 来做两类事：

(A) 预防错误/危险操作（限制它能做什么）

例子：

- 禁止执行 `rm -rf /` 这种命令
- 禁止访问某些敏感路径（如 `~/.ssh`）
- 限制网络访问（避免乱装包/乱发请求）

(B) 检测并纠正“低质量动作”（约束它怎么做）

例子：

- **语法检查**：它生成的 patch 不能连 Python 语法都错
- **单元测试门槛**：改完必须跑测试，失败就不允许继续提交

比如规则：一次改动不能改太多文件，不能改关键模块

确实，这个AI都会乱回答问题，很难不保证这个AI智能体会乱使用电脑

这里的 **debug** 你可以理解成：“把代码里导致错误/不符合预期的地方找出来并修好”。
它不等于“写新功能”，更像“修理工模式”。

1) debug 在软件工程里具体做什么？

一般包含 3 步闭环（很像机器人“感知-行动-再感知”）：

- 1. **复现问题 (reproduce)**：让错误稳定出现
- 2. **定位原因 (localize)**：是哪一行/哪个逻辑导致的（根因 root cause）
- 3. **修复并验证 (fix & verify)**：改代码 + 跑测试/样例，确认真的好了

2) “短代码 debug 任务”是什么意思？（HumanEvalFix 的语境）

HumanEval 你可以当成“写函数小题库”。

HumanEvalFix 则是：**给你一段短函数代码，它是“错的”或“有 buq 的”，你要修到通过测试。**

这个debug可以理解为修复漏洞的过程

一句话串起来（你读论文时的“框架记忆”）

- **Ablation**：证明哪些模块有用
- **Shell-only baseline**：用最朴素方法当对照
- **Config search**：自动调参找最好组合
- **Window size**：一次给模型看的上下文长度
- **Temperature**：生成时的随机性开关

如果你愿意，把论文里对应的实验表（比如 ablation 表、参数设置表）再截一张，我可以带你“按表读论文”：每一行到底在控制什么、为什么会提升/下降。

📄 🍷 🔍 ↺ ...

🔑 **核心观点**: LM 是一类“新用户”，需要专门为其设计的接口（ACI），就像人类需要 UI/GUI 一样。

🕒 ACI 设计四大原则

原则	解释	类比
1. 动作简单易懂	命令选项要少，文档要精简	像给小孩用的简化版工具
2. 动作紧凑高效	一个命令完成一个完整操作	不要让 LM 做“连续动作组合技”

好一个将这个语言模型看做是一个新用户

egrated. (a) The file viewer shows the agent the
nvokes the edit function to replace lines 404-407
the agent the now updated version of the file.

“随口输出一段代码”

传统让 LLM 修 bug 容易变成。

- 它输出一大段“改后的完整文件”
- 或者输出一个不清晰的“你把 if 改一下”
这样很难自动执行，也难以审计。

而这里的 ACI 做到了：

1. 观察带行号（可定位）
2. 动作带范围（可控、最小修改）
3. 修改后可立即重新观察更新版文件（闭环反馈）

这就像给机器人一个“抓取原语”：

不是让它输出每个电机电压，而是让它说“抓住这个物体”，系统负责把动作落地。

你应该从这张图记住的 3 个关键词

关键词

你可以怎么理解

感觉这个仅仅是将这个智能体操作规范化了，就可以发一篇文章，但是这个规范化的代码应该不是很好写的，但是在当前2026年，这个借助claude和codex应该也变得容易了